

本科毕业设计 · 2026

基于多 Agent 协同的 LLM 驱动复杂业务软件 自动化生成的研究与实现

从 ChatBot 到 Agent：让 AI 真正参与软件工程全流程

温俊杰 202230500125

答辩分享 · ~ 15 min

01 · 项目与背景 — jcli · AI 终端工作站

jcli · 主项目 · Rust · 5.2 万行

多模型终端 Agent。包含上下文治理（分级压缩 + 优先级窗口 + 动态 System Prompt）、面向场景强化的工具（Plan / Edit / 执行三态 / system-reminder）、多 Agent 协作（Teammate + SubAgent），以及 Skill / Hook 工程化扩展机制。

jcli · 主项目 github.com/LingoJack/jcli

jen · 模板代码引擎 · Go · model_infrax

自研 SQL parser + Go template，将 SQL schema 确定性生成 PO/DTO/VO/DAO 全套方法。按主键 / 唯一索引 / 普通索引分别派生查询方法，消除 LLM 长链路推理中的字段漂移。封装为 Skill，按需加载。

jen · 模板代码引擎 github.com/LingoJack/model_infrax

artifact_demo · 端到端验证制品

用 jcli 配合 webapp-gen 和 jen 从零生成的电商 Web App（React + Go + GORM + Podman），三环境矩阵验收全部通过。

artifact_demo · 验证制品 github.com/LingoJack/gradiation_artifact_demo

三个仓库均已开源 · MIT License

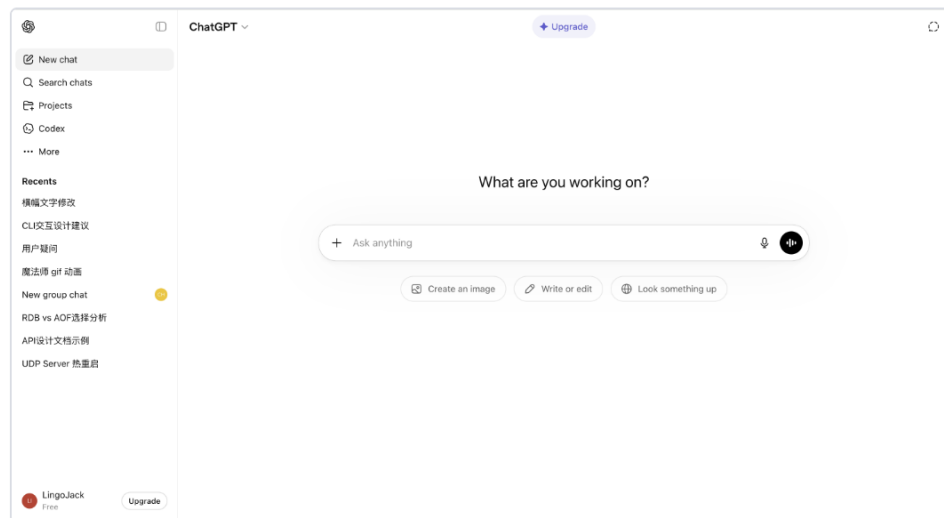


01 · 项目与背景 — 今天要讲的 6 个部分

01	项目与背景	~4min
02	上下文治理与状态感知	~6min
03	面向场景强化的工具	~8min
04	多 Agent 协作试验	~5min
05	模板代码生成引擎	~4min
06	总结与展望	~3min

01 · 项目与背景 — ChatBot 与 Agent 的上下文成本对比

ChatBot



- 人手收集代码、报错、配置
- 复制粘贴到对话窗口
- AI 给建议，人手动改代码
- 人承担上下文收集和执行反馈
- 本质：人是物理/逻辑世界的桥梁

Agent



- Agent 通过工具主动读取、执行
- Agent 自动管理上下文窗口
- Agent 自主决策、执行、纠错
- Agent 承担上下文治理和工具调用
- 本质：Agent 成为桥梁，人抽离

01 · 项目与背景 — Function Calling: LLM 如何调用工具

LLM 是文本预测器，不能直接操作文件或终端。**Function Calling** 是"说话"到"做事"的桥梁。

LLM 的输入结构

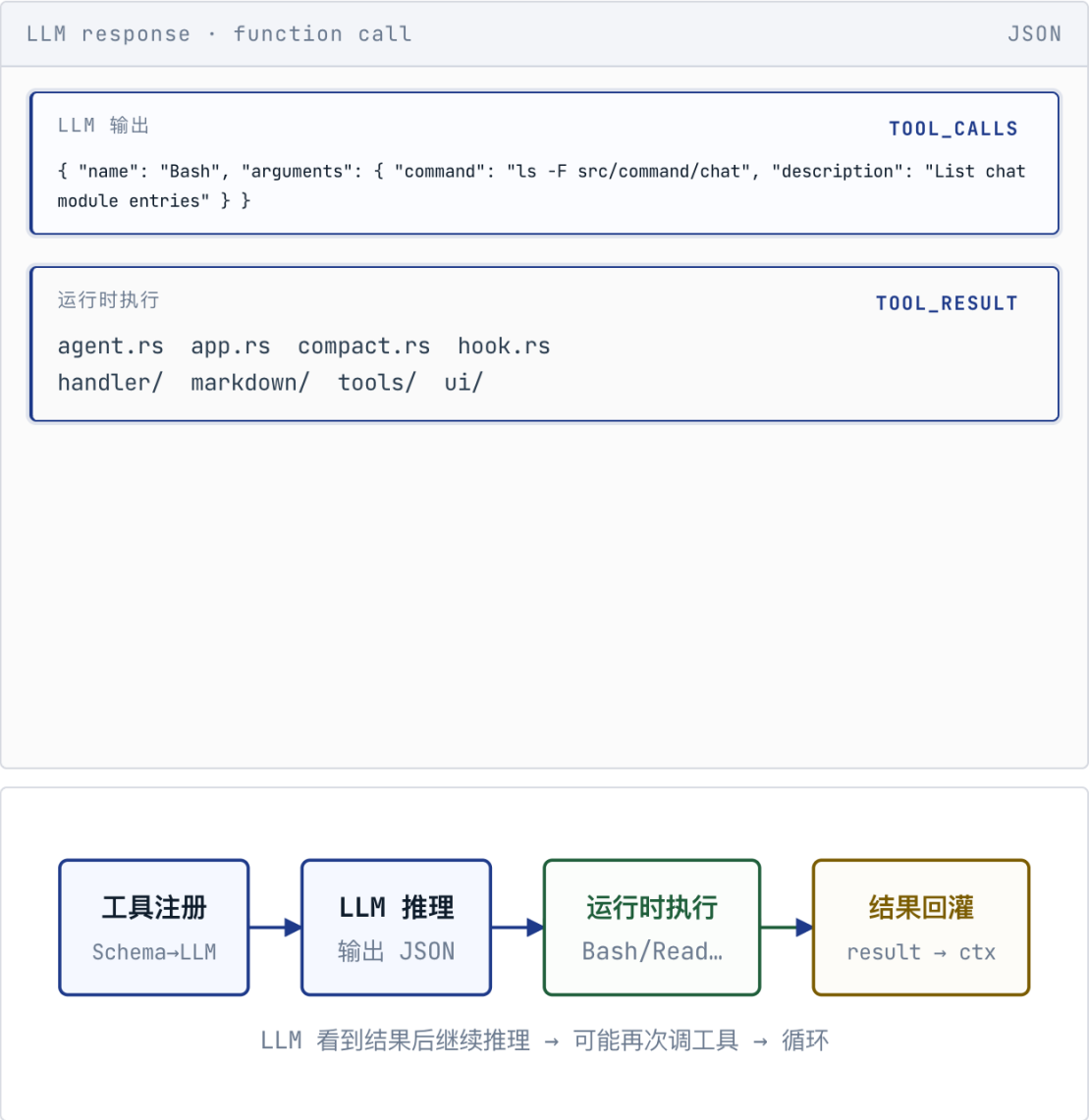
每次请求，LLM 收到一份 **system prompt** + 一组 **messages**：

- **User** · 用户输入 + 系统注入的提醒
- **Assistant** · LLM 回复，含文本和 `tool_calls`
- **Tool Result** · 工具执行结果回灌

Function Calling 流程

- **工具注册** · 名称、描述、参数 Schema 写入 System Prompt
- **LLM 决策** · 输出 `tool_calls` JSON (工具名 + 参数)
- **运行时执行** · 解析 JSON，执行工具，结果回灌 messages
- **循环迭代** · LLM 看到结果后继续推理，可能再次调工具

关键：LLM 只产出"意图"，运行时代替它执行。



01 · 项目与背景 — ReAct 核心循环

Agent 不是一次推理出全部答案。它**逐轮思考、调工具、读结果、再思考**。

- **Thought** · 读当前上下文，决定下一步
- **Action** · 输出符合 JSON Schema 的 tool call
- **Observation** · 运行时执行工具，把结果序列化回灌窗口
- **Update** · 窗口变大，进入下一轮 Thought

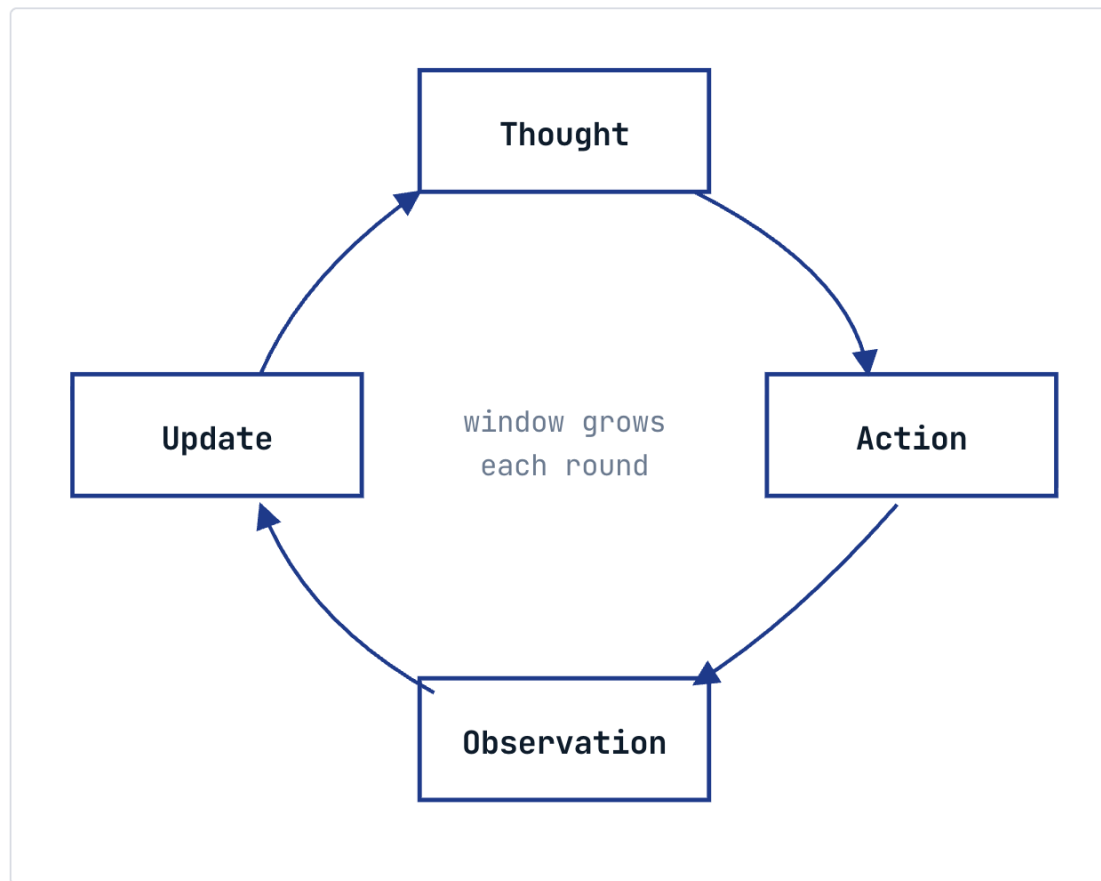


图 1 · ReAct 单轮循环

接下来六页用一个真实任务演示这个循环 —— 同时观察上下文窗口是怎么被填满的。

01 · 项目与背景 — ReAct 逐帧演示

用户的需求**不是机器指令**。Agent 需要先把它**翻译成**可执行的工具调用。

- "总结" → 需要看目录结构
- "可优化的模块划分" → 需要读关键源文件
- 用户没说去哪儿看，Agent 自己定位

此时窗口里只有：System prompt + 用户这一句话。

jcli chat · session #42~/dev/j

USER@ME

总结一下 src/command/chat 目录的结构，以及可以优化的模块划分。

context 6%

state:

Thought

Action

Observation

Update

01 · 项目与背景 — ReAct 逐帧演示

Agent 先列目录看全貌 —— 这是面对未知项目最常见的第一步。

- 选择工具 `Bash`，因为要枚举文件
- 参数符合 JSON Schema，运行时能直接解析
- 用 `ls -F` 区分文件与目录

注意：Agent 的“想法”也被写进上下文窗口 —— 它是后续轮次的决策依据。

jcli chat · session #42~/dev/j

USER

总结一下 src/command/chat 目录的结构.....

AGENT

THOUGHT

用户要总结目录结构，先列出 chat/ 下的文件分布。

TOOL CALL

BASH

command: ls -F src/command/chat

description: List chat module entries

context

9%

state:

Thought

Action

Observation

Update

01 · 项目与背景 — ReAct 逐帧演示

运行时在真实 shell 里执行 tool call，把 stdout 序列化为 tool-result 消息，写回上下文。

- 12 个条目，足够建立粗略地图
- Agent 在下一轮能看见这些文件名
- 结果以 文本形式 永久落在窗口里

jcli chat · session #42

~/dev/j

AGENT

THOUGHT

用户要总结目录结构，先列出 chat/ 下的文件分布。

TOOL CALL

BASH

command: ls -F src/command/chat

TOOL RESULT

STDOUT

agent.rs handler/

api.rs input_thread.rs

app.rs markdown/

archive.rs mod.rs

autocomplete.rs permission.rs

compact.rs render_cache.rs

constants.rs skill.rs

hook.rs storage.rs

theme.rs

tools/

ui/

ui_helpers.rs

context

14%

state:

Thought

Action

Observation

Update

这是 ReAct 的关键：tool 输出是会长期占用 token 的，不是临时变量。

01 · 项目与背景 — ReAct 逐帧演示

Agent 看完目录，决定深入 `app.rs` —— 它是名字最像入口的文件。

- 选 Read 而非 Bash cat —— 结构化输出、自动行号
- 用 offset/limit 控量，不读全文
- 结果会很长，是窗口压力的主要来源

jcli chat · session #42

~/dev/j

TOOL RESULT

agent.rs app.rs compact.rs hook.rs ...

AGENT

THOUGHT

app.rs 看起来是顶层入口，先读它的 struct 定义。

TOOL CALL

READ

file_path: src/command/chat/app.rs

limit: 60

TOOL RESULT

60 LINES

pub struct ChatApp {
 pub ui: UIState, // 55 fields
 pub state: ChatState, // 7 fields
 pub tool_executor: ToolExecutor,
 pub agent: Option<AgentHandle>,
 pub tool_registry: Arc<ToolRegistry>,
 pub jcli_config: Arc<JcliConfig>,
 ...
}

context

24%

state:

Thought

Action

Observation

Update

真实 Read 一次大约带回 60 行源码 (800–1500 tokens)。

01 · 项目与背景 — ReAct 逐帧演示

经过 3 轮工具调用 (ls + Read app.rs + Grep "pub struct"), Agent 拼出全貌。

- 窗口最终占用 **38%**
- 原始用户问题仅 1 行，结论却用了几千 token 才得到
- 价值密度低 —— 大量字符是 **中间产物**

jcli chat · session #42

round 3 done

TOOL RESULT

ROUND 2 · READ

pub struct ChatApp { ... } · 55+7+9+2 fields ...

TOOL RESULT

ROUND 3 · GREP "PUB STRUCT"

app.rs:24 ChatApp · app.rs:108 UIState · app.rs:312 ChatState · ...

AGENT

FINAL

src/command/chat 共 27 个文件，按职责分四层：
1. 入口 · mod.rs / app.rs / agent.rs
2. 状态 · UIState (55 字段) + ChatState (7 字段)
3. I/O · handler/ · input_thread · render_cache
4. 扩展 · tools/ · skill · hook · compact

可优化点：UIState 字段过多，可按"输入/popup/缓存"再切分。

context

38%

state:

round 1

round 2

round 3

summarize

下一节·上下文窗口治理：怎么让这些"中间产物"不污染长程任务。

上下文治理与状态感知

LLM 的窗口是物理硬上限。Agent 要长期运行，必须有一套机制来管理"往窗口里塞什么、丢掉什么"。

上一章的 ReAct 循环已经展示：每轮工具调用的结果都会永久占用上下文 token。轮次越多，窗口越满，Agent 开始幻觉、重复操作、忘记目标。

两条治理路径

- 分级压缩 · micro 快速替换旧工具结果为占位符，auto 调用 LLM 做深度摘要。均配备豁免机制保护关键信息。
- 优先级窗口 · 不是固定取最近 N 条，而是给消息打价值分，按 3:6:1 比例填充，保证方向性信息优先入窗。实测可稳定运行 2000+ 轮。



图 · 窗口压力来源与两条治理路径

02 · 上下文治理与状态感知 — 分级压缩与豁免机制

Micro 压缩 · 快速替换

将久远的工具调用结果替换为占位符，仅保留"调用了什么"而丢弃内容。速度快、token 释放大，适合窗口接近上限时紧急腾空间。

Auto 压缩 · 深度摘要

调用 LLM 将对话历史总结为精简摘要，然后清空窗口 + 写入摘要。成本较高但压缩彻底，适合 Agent 运行很久需要"大扫除"的场景。

豁免机制 · 保护关键信息

无论 micro 还是 auto，以下消息不被压缩：

- 用户消息 · 需求源头
- Agent 纯文本回复 · 决策记录
- TodoWrite / Plan / Skill · 方向性信息
- SubAgent 最终结果 · 调研结论

工具结果的价值衰减

工具类型	价值判断	豁免
Read	低 · 文件内容随编辑失效	No
Bash	低 · 输出过时极快	No
Edit	低 · diff 已被后续覆盖	No
Write	低 · 与 Edit 同理	No
Grep	低 · 搜索结果一次性	No
TodoWrite	高 · 任务追踪	Yes
Plan	高 · 执行方案	Yes
Ask	高 · 用户意图确认	Yes
Skill	高 · 能力定义	Yes

Bash / Edit / Read / Write 的信息价值随轮次衰减极快——第 20 轮时，第 3 轮的 Read 结果对决策几乎无价值。

02 · 上下文治理与状态感知 — 优先级窗口：3:6:1 价值比例填充

固定取最近 N 条消息的问题：方向性信息可能在很早的位置，被新消息挤出窗口。我们把上下文当作推荐系统——给消息打价值分，优先填高价值的。

高价值消息类型

- 用户消息 · 需求源头，方向锚点
- TodoWrite · 当前任务追踪
- Clean (Plan clear-and-approve) · 执行方案
- Skill 加载 · 能力定义
- Agent 纯文本回复 · 决策记录

3:6:1 填充比例

在固定窗口内，按 3:6:1 比例分别加载高价值 / 中价值 / 低价值消息。保证 Agent 在长程运行时始终能看到方向性信息。

实测：可稳定运行 2000+ 轮消息持续任务，Agent 方向性保持稳定。



02 · 上下文治理与状态感知 — 动态状态感知：两层注入机制

问题

Agent 每轮推理只能看到上下文窗口内的内容。一旦信息被压缩或滚出窗口，Agent 就失去感知——忘记后台任务、忽略待办、遗漏 Teammate 消息。

两层注入

- **System Prompt 占位符** · 模板中 `{{.background_tasks}}`、`{{.teammates}}` 等在每轮请求前替换为实时状态快照
- **<system-reminder> 消息** · 突发事件（后台任务完成 / TODO 过期 / Teammate 新消息）以 User 角色按需注入

软提醒，不强打断——Agent 在下一轮推理时自然看到，可暂时忽略处理更紧急的事。

静态基础	项目规范 AGENTS.md	工具定义 Tool Schema	环境信息 OS / Shell	Skill 按需加载
LAYER 1 · SYSTEM PROMPT 占位符	后台任务 Task / BackgroundJob	Teammate 协作队友状态	TODO 过期未更新提醒	Session 交互式任务
LAYER 2 · SYSTEM- REMINDER 消息	任务完成 background_task_completed	TODO 过期 todo_reminder	队友消息 teammate 新消息	Skill 重新注入 invoked_skills

02 · 上下文治理与状态感知 — 治理成果：长程运行实测

三重保障的协同效果

- 分级压缩 · micro 快速替换旧工具结果，auto 深度摘要清空窗口
- 优先级窗口 · 3:6:1 比例保证方向性信息始终在窗
- 动态 System Prompt · 压缩后仍感知持续性事件

指数退避 · API 鲁棒性

LLM API 限流、超时由指数退避 + 随机等待自动恢复，不打断工作流。

使用场景：睡前审批 Plan → Agent 自主执行 → 早上收到可运行原型。



面向场景强化的工具

每个工具的引入都是为了解决一个具体问题——Grep 构建上下文、Ask 防漂移、Plan 先调研再执行、Edit 精确替换、执行三态自动流转。

03 · 面向场景强化的工具 — 从最小 Agent 到渐进增强

最小 Agent：4 个工具即可工作

Bash + Read + Write + LoadSkill — 能读文件、写文件、执行命令、按需加载技能。理论上够用。

问题：基础工具覆盖不了的场景

- 面对未知项目，只能 `Bash grep` 搜索 → 匹配质量不稳定、格式难解析
- 用户需求模糊 → 盲目执行，产物与预期**漂移**
- LLM 没想清楚就开始改 → 变更不完整，调研占满窗口
- 长任务跑着跑着忘记目标 → 漏改漏做

每个增强工具都是为了解决一个具体问题。

基础层 = 最小 Agent | 增强层 ← 每个工具解决一个具体问题

基础层	Read 读取文件	Write 写入文件	Bash 执行命令	LoadSkill 按需加载
增强层	Grep 模板匹配	Ask 澄清意图	Plan 先调研再执行	Web 网络搜索
协作层	Todo 任务追踪	SubAgent 委派调研	Teammate 多人协作	Session 交互会话

03 · 面向场景强化的工具 — Plan 调研 + 审批清空上下文

Plan — 先调研再执行

问题：LLM 边想边改，调研读 30 个文件占满窗口 → 改一处忘一处。

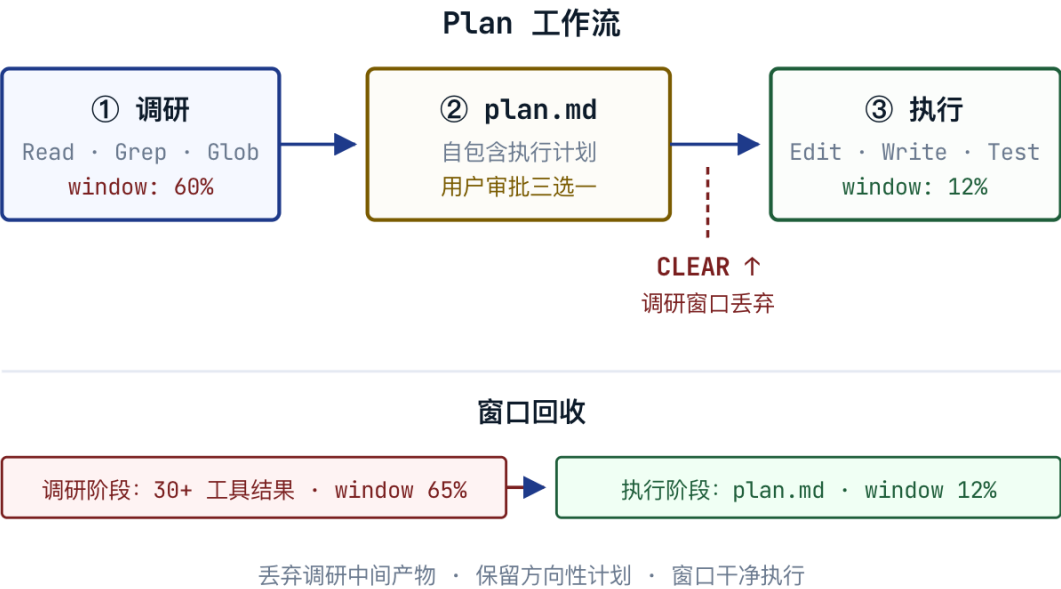
方案：Plan 模式 → 只读调研 → plan.md → 审批 → 清空上下文 → 按计划执行。

审批三选项

- ① 批准 — 保留上下文，开始实施
- ② 批准并清空上下文 — 仅保留 plan.md，丢弃调研产物 ← 推荐
- ③ 驳回 — 留在 Plan Mode 修改

窗口回收效果

调研阶段大量工具结果（Read / Grep / Bash）在执行阶段**全部丢弃**，仅保留一份 plan.md（~1.2K tokens）。窗口从 65% → 12%，执行阶段拥有充足空间。



03 · 面向场景强化的工具 — 清空窗口按计划执行

窗口从 65% 回到 12%：调研产生的 30+ 工具结果全部丢弃。

- Agent 重新启动，上下文只含 plan.md
- 工具集恢复为完整集（Edit / Write / Bash）
- 不再有调研中间产物干扰决策

clear & approve 机制：忘掉调研细节 → 窗口干净，错误率下降

Sprite context(36K) · message(405) · 执行 ExitPlanMode...

SubAgents: Research_actuol_tool_ [✓ 已完成 t84]

chat-module-guide, fullstack-team, html-ppt, hyperframes, j-cli, jcli-dev-guide, skill-creator, sql-to-go-struct-and-dao, swift-ios-app-gen, ui-ux-pro-max, webapp-gen

需要补充 8 个技能

o 执行计划

步骤 1: 更新 commands.md (中英文)

补充 6 个缺失的彩虹命令 (/clear , /theme , /resume , /dump , /dump-processed , /leammate)

使用代码中已有的描述文本

步骤 2: 更新 tools.md (中英文)

编辑: 添加 replace_all 和 confirm_token 参数

Glob: 添加 offset 参数

Bash: 添加 description 和 interactive 参数

Agent: 添加 worktree 和 inherit_permissions 参数

新增工具章节: Session, Teammate, SendMessage, LoadTool, EnterWorktree, ExitWorktree

步骤 3: 更新 skills.md (中英文)

tools.md 中 LoadSkill 部分的能力列表需要补充 8 个缺失技能

skills.md 本身主要是结构说明, 无需修改

o 文件修改清单

文件	修改类型
web/src/data/docs/zh/commands.md	补充命令
web/src/data/docs/en/commands.md	补充命令
web/src/data/docs/zh/tools.md	补充参数 + 新增工具
web/src/data/docs/en/tools.md	补充参数 + 新增工具

o 注意事项

1. 保持现有文档格式风格 (表格 + JSON 示例)

2. 中英文版本同步更新

3. 描述请提取自代码中的定义, 确保准确

4. 参数类型使用一致的命名 (string/uint/bool/array/object)

o 批准计划

批准此计划, 保留当前上下文, 开始实施

o 批准并清空上下文

批准计划并清空探索过程中的对话上下文, 仅保留计划内容继续实施

o 驳回计划

拒绝此计划, 留在 Plan Mode 中修改方案

o 自由输入...

Up/Down Move | Enter OK | PgUp/PgDn Scroll | Esc Cancel

+ 补充消息, Enter 发送, Esc 打断, Alt+Enter 换行

选择 | Enter 确认 | Esc 拒绝

jcli plan-mode · 批准 / 清空上下文 / 驳回三选项

execute-mode · session #78 (new ctx)

full tools

SYSTEM

CONTEXT CLEARED

上下文已清空。仅保留 plan.md (~1.2K tokens)。可使用完整工具集。

AGENT

STEP 1/5

按 plan.md step 1: 创建 ui_state.rs。

TOOL CALL

WRITE

file_path: src/command/chat/ui_state.rs

content: pub struct UIState { /* 55 fields */ }

TOOL RESULT

created 89 lines

AGENT

STEP 2/5

step 1 done. 进行 step 2: 创建 chat_state.rs。

context

12% · clean window

03 · 面向场景强化的工具 — Edit：精确替换与凭证确认的批量机制

精确字符串匹配

基于 `old_string` 在文件中做精确匹配，替换为 `new_string`。只发送变更部分，不触碰文件其他内容。匹配失败时返回**诊断提示**（缩进、空白、行尾差异），帮助快速定位问题。

唯一性约束

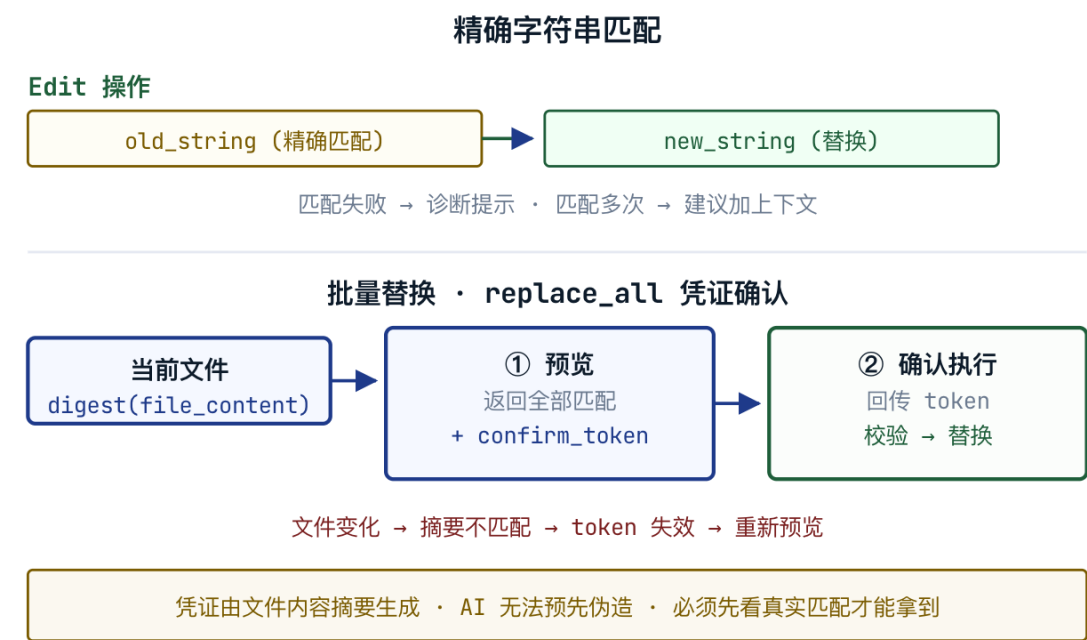
`old_string` 在文件中必须**唯一匹配**。匹配多次时返回带上下文的预览，提示加更多上下文使其唯一。

批量替换 · 凭证确认机制

当 `replace_all=true` 时，基于当前文件内容生成摘要凭证，强制先预览再执行：

- ① **预览** — 返回全部匹配位置 + `confirm_token`（由文件内容摘要生成）
- ② **确认** — 回传 token 才执行替换。文件内容变化 → 摘要不匹配 → 凭证失效 → 强制重新预览

凭证绑定文件快照，AI 无法预先伪造——必须先看到真实匹配才能拿到凭证。



03 · 面向场景强化的工具 — 执行三态概览

问题背景 · 一个真实的卡死案例

生成业务软件时，Agent 执行了 `npm create vite@latest my-app -- --template react-ts`。命令久久没有返回

排查发现：这个命令有交互式设计，要求用户输入 Enter 确认安装。但 Agent 用的是同步 Bash —— 进程在等 stdin，Agent 在等 stdout，双方互相等待，形成死锁。

这类交互式命令在包管理器中很常见：npm init、pip install、apt-get、Homebrew.....Agent 必须能识别并正确处理。

设计目标

入口统一 —— Agent 调用的都是 `Bash` 工具。运行时自动识别命令特性，落到对的执行形态：

- Bash · 同步 · 短命令阻塞返回
- BackgroundTask · 异步 · 长命令后台运行
- Session · 交互 · 需要 stdin 的命令走 PTY

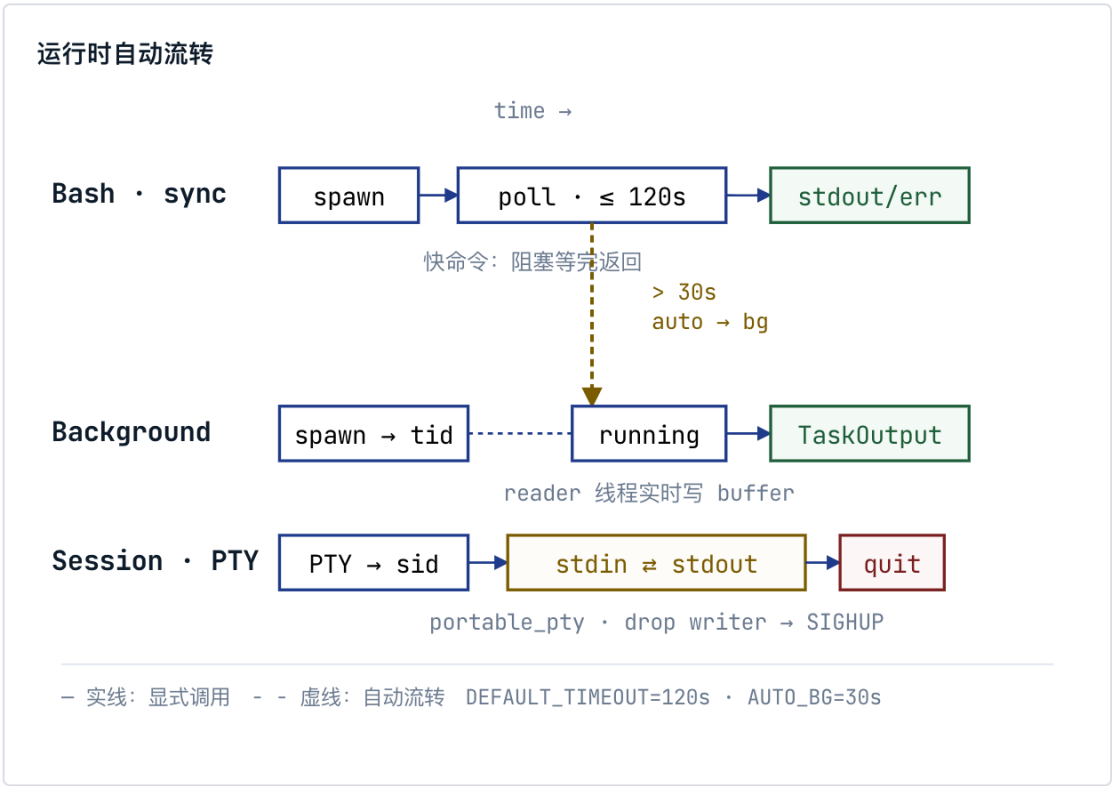


图 · 执行三态：横轴时间，纵向三泳道，虚线箭头为自动流转

03 · 面向场景强化的工具 — 执行三态实现

① 超时自动升级

同步 Bash 跑过 `SHELL_AUTO_BG_SECS` (30s) 仍未结束:

- 静默检测 — N 秒无输出 → 疑似交互 → kill + 提示
- 非静默 → 不 kill, `adopt_process` 移交 BackgroundManager
- 立即返回 `task_id`, Agent 继续做别的事
- 后台监控线程等 reader 写完后调 `complete_task`

② PTY 交互式会话

用 `portable_pty` 分配伪终端:

- `openpty` → `spawn_command` 启动进程
- `try_clone_reader` → Session stdout
- `take_writer` → `set_pty_writer` 注册到 BgTask
- Session 工具的 `stdin/stdout/quit` 直接操作 PTY 句柄
- quit = drop writer → SIGHUP 自然退出

shell.rs · execute_sync 自动升级

real code

```
// 超过阈值仍未结束, 不 kill, 移交后台
if start.elapsed() >= auto_bg_threshold {
    // 静默判交互: 最近 N 秒无输出 → 等 stdin
    if silence_elapsed >= Duration::from_secs(SILENCE) {
        let _ = child.kill(); // 提示加 --yes
        return ToolResult { output: hint, is_error: true, .. };
    }
    // 真正升级: adopt 当前进程, 立即返回 task_id
    let (task_id, bg_buffer) =
        self.manager.adopt_process(command, pid, start);
    // 后台监控: 等 reader 写完 → complete_task
    std::thread::spawn(move || {
        let _ = stdout_thread.join();
        manager.complete_task(&tid, "completed", out);
    });
    return ToolResult { output: json!({ "task_id": task_id }), .. };
}
```

shell.rs · execute_interactive 起 PTY

real code

```
let pty = NativePtySystem::default();
let pair = pty.openpty(PtySize { rows: 24, cols: 80, .. });
let _child = pair.slave.spawn_command(builder)?;
let reader = pair.master.try_clone_reader()?; // stdout
let writer = pair.master.take_writer()?;      // stdin
self.manager.set_pty_writer(&task_id, writer);
// Session 工具通过 sid 调 stdin / stdout / quit
```

多 Agent 协作

两种协作模式——Teammate 对等协作 + SubAgent 委派调研。共享消息通道的设计、可见性控制、以及三个递进实验验证。

04 · 多 Agent 协作 — 两种协作模式

Teammate · 对等协作

独立 Agent 实例，各有专属上下文和工具集。通过 `SendMessage` 广播 + `@提及` 定向通信。支持 `WorkDone` 声明完成，`@self` 可重新激活。多 Teammate 共享消息通道，形成聊天室。

SubAgent · 委派调研

临时子 Agent，独立窗口完全隔离。无收件箱、不能发消息。执行完毕后仅回灌精简摘要（~1.5K tokens）到主 Agent 上下文，主窗口仅微增 4%。

客观评估

参考 Anthropic 研究——模型倾向偏袒自己的工作。通过聊天室让其他 Agent 交叉评审，实现相对客观的质量评估。

TEAMMATE 协作架构



SUBAGENT 委派架构



04 · 多 Agent 协作 — 消息通信设计与可见性

三通道架构

- `broadcast_inbox` — 每个 Teammate 的收件箱，其他 Agent 广播投递
- `context_messages` — 注入 Main Agent LLM 上下文（XML-wrapped `<From>text</From>`）
- `display_messages` — TUI 渲染（clean text + sender_name）

设计动机

Agent Loop 长时序异步，Agent 发完消息不原地等确认 → 下一轮可能因过期快照重发。这一问题的实验验证与解决方案（SendMessage Gate）在末页详述。

消息可见性矩阵

	Main Agent	Teammate	SubAgent
Own messages	Full	Full	Full
Other agent text	context (XML)	broadcast_inbox	None
Other tool calls	<Name>[called X]</Name>	Not visible	None
Other tool results	Not pushed	Not visible	None
Can SendMessage	Yes	Yes	No
Can receive	main_inbox	broadcast_inbox	No

设计原则: 决策信息多放（other agent text），执行中间过程少放（tool results 不推）。SubAgent 完全隔离——不能收发消息，只回灌摘要。

04 · 多 Agent 协作试验 — 轮流报数：过期快照问题

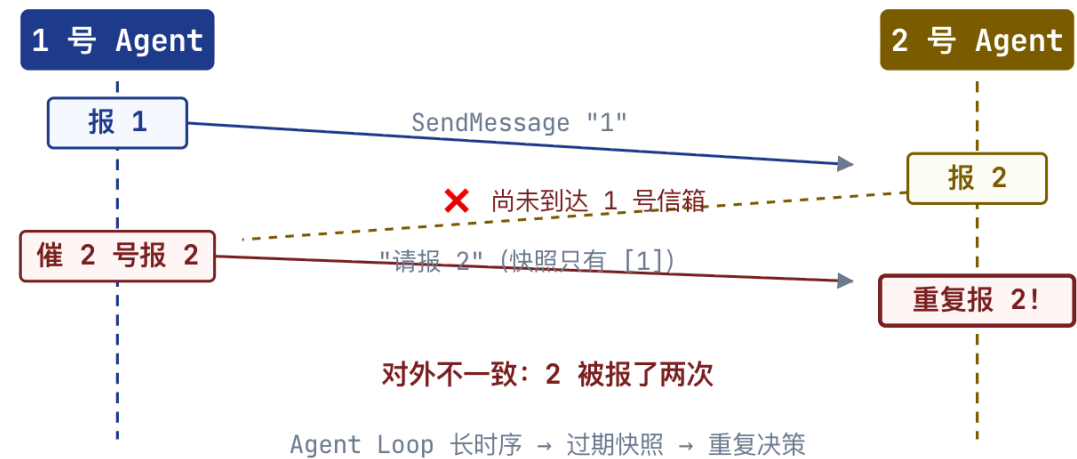
实验内容

两个 Teammate 轮流报递增数字 (1, 2, 3 ...), 验证多 Agent 之间的消息传递与唤醒能力。仅通过 SendMessage 沟通, 不共享全局状态。

发现的问题 · 过期快照导致重复消息

1 号 Agent 进入循环时, 2 号已报的"2"尚未到达其信箱。1 号快照只有 [1], 于是催促 2 号报 2 → 2 号看到 [1,2] + 催促 → 重复报 2 → 对外不一致。

根本原因: Agent Loop 长时序异步, 发完 SendMessage 不原地等确认 → 下一轮基于过期快照决策。



04 · 多 Agent 协作试验 — 二阶段提交 Gate 与实测结果

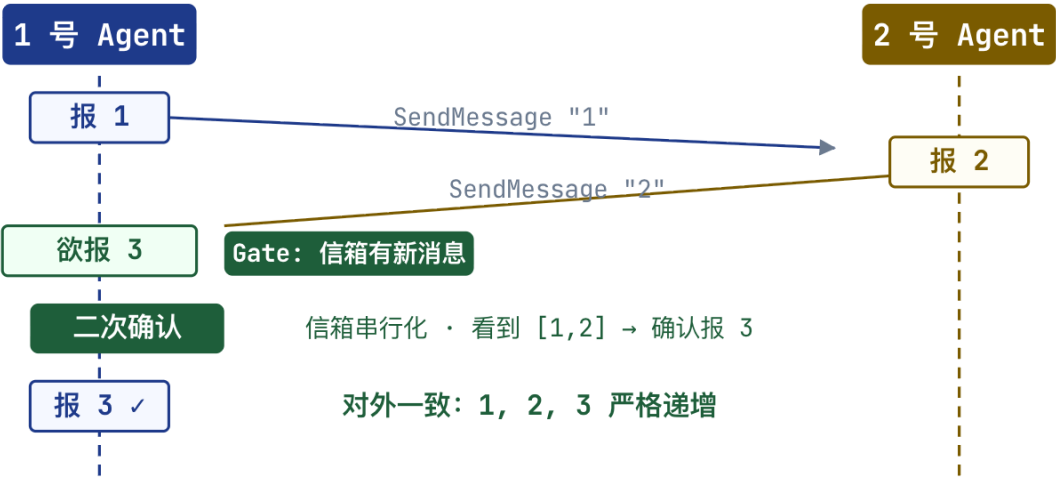
解决 · SendMessage Gate (二阶段提交)

Agent 调 SendMessage 前, 先检查 `broadcast_inbox` 有无未读消息。有 → hold + 注入 `system_reminder` → 信箱串行化 → Agent 基于最新状态二次确认 (commit 或重新决策), 最多重试 2 次。

```
teammate_loop.rs · SendMessage Gate
if item.name == "SendMessage" {
    let inbox_has_new = inbox.lock()
      .map(|i| !i.is_empty()).unwrap_or(false);
    if inbox_has_new && retries < MAX_GATE_RETRIES {
        gate_triggered = true;
        // hold: return placeholder, inject system_reminder
        continue;
    }
}
```

实测结果

目标数字数	10
正确报数数	10 / 10 · 100%
顺序错误数	0
唤醒失败数	0
SendMessage 调用	16 次
人工干预次数	0



04 · 多 Agent 协作试验 — 共享上下文一致性

任务设置

多个 Teammate 轮流续写一句故事，保持人物、地点、时间线和事件因果一致。

检验目标

实验一只验证"能按序发言"，本实验进一步检验**共享上下文**——每个 Agent 是否真能读到前文设定。如果 shared_messages 广播有 bug，故事会立即跑题。

验证方式

统计完成轮数、偏题次数、重复内容次数、人工干预次数。

论文表 7-11 · 实测结果

目标续写轮数	13
完成续写轮数	13 / 13 · 100%
偏题次数	0
重复内容次数	1
人工干预次数	0

共享上下文有效：13 轮全部完成，偏题 0、人工干预 0。仅 1 次重复——角色「反转之王」被催促后重发了同句，提示仍需消息去重 / 过期约束。

04 · 多 Agent 协作试验 — 开发-测试协作

What · 任务设置

Developer + Tester 双角色协作。任务是给前面端到端生成的电商网页项目**追加 45 项验收清单**（A 类页面跳转 20 + B 类浏览历史 15 + C 类 UX 改进 10）。Tester 把需求拆为可验收项 → 跑构建/静态检查 → SendMessage 反馈失败项 → Developer 修复 → 复验。

Why · 客观评估的关键

参考 Anthropic *harness-design-long-running-apps*: **模型对自己的工作有偏袒**，倾向给自己打高分。让**另一个 Agent 做评审**才能得到相对客观的反馈，形成"实现—验证—反馈—修复—复验"的工程闭环。

论文表 7-13 · 实测结果

A 类 · 页面跳转	20 / 20
B 类 · 浏览历史	15 / 15
C 类 · UX 改进	10 / 10
需求完成率	45 / 45 · 100%
第 1 轮验收	41 通过 / 4 失败
第 2 轮复验	45 / 45 全过
Tester 反馈轮次	2 轮
Developer 修复轮次	1 轮
协同冲突 / 重复劳动	0 / 0
人工干预	2 次（仅流程调度）

闭环完整：需求清单 → 实现 → 首轮验收 → 反馈 → 集中修复 → 复验通过。Tester 发现的 4 个失败项都是局部细节（字段缺失、路径不一致等），未涉及架构级偏差。

04 · 多 Agent 协作 — SubAgent：低上下文成本的调研委派

上下文隔离

独立消息窗口，不继承父 Agent 历史。以 `prompt` 为唯一起点，在独立窗口里 Read / Grep / WebFetch。

工具隔离

独立 `ToolRegistry`，排除 `Teammate` / `Agent` 工具（防递归）。可选 `worktree` 创建独立 git 工作树。

结果回灌

执行完毕后，仅回灌精简摘要到主 Agent：

- `display` — clean text + sender_name
- `context` — `<SubAgent@Name>摘要</SubAgent@Name>`
- 工具结果**不推入** context（token 高、价值低）



模板代码生成引擎

把高度模板化的代码交给确定性生成器，LLM 只做需求理解和增量集成。jen 处理数据层，webapp-gen 驱动全栈工作流。

05 · 模板代码生成引擎 — 问题

让 LLM 写模板代码的代价

业务系统中存在大量高度模板化的代码——PO / DTO / VO / DAO、CRUD 方法、列表查询。它们占比高，但业务含量有限。让 LLM 逐字生成，会怎样？

51 张表 DAO 生成实测（论文 §7.4）

- 11 个字段遗漏 · 247 / 258，字段一致性仅 95.7%
- 4 个索引方法缺失 · 22 / 26，写到第 30 张表时已忘了第 5 张的唯一索引
- 24M token · 53 min · LLM 调用 582 次，工具调用 2,006 次
- 67 次上下文压缩 · 长链路推理撑爆上下文窗口

本质

概率性的长链路推理的系统性风险——模板代码字段多、重复度高，累积误差不可避免。

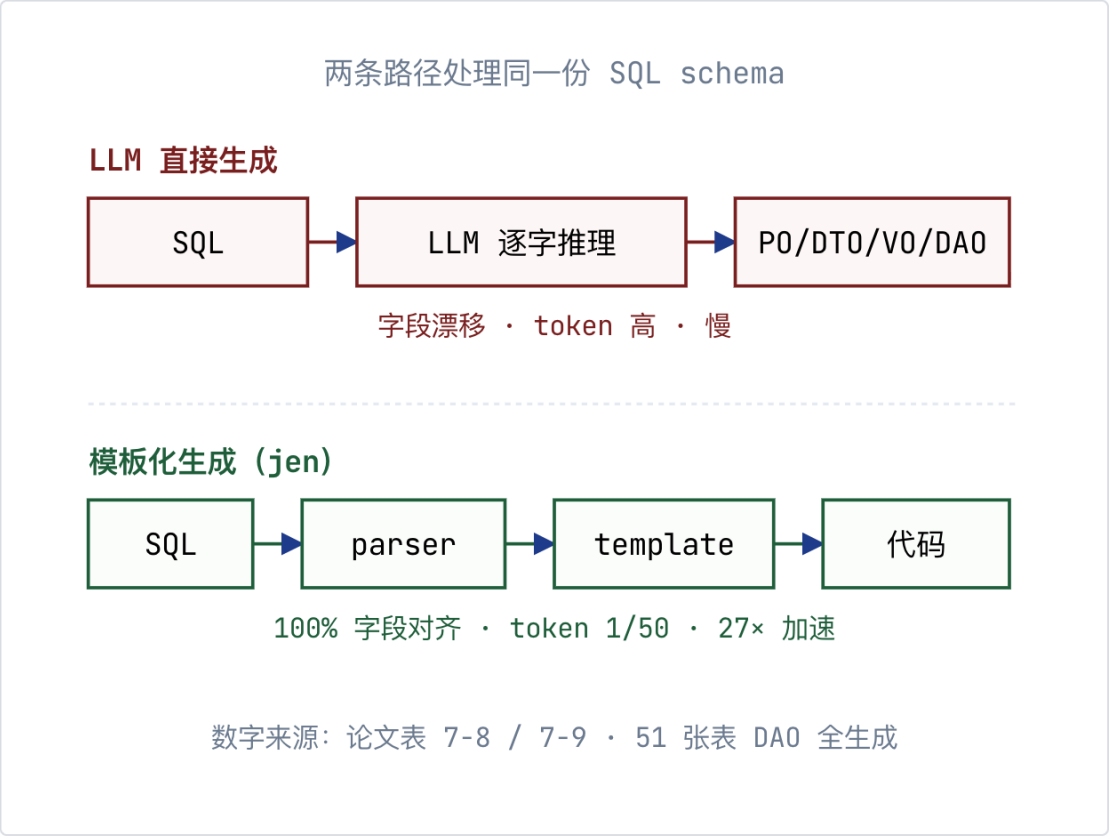


图 · 模板化生成 vs LLM 现场生成

05 · 模板代码生成引擎 — 核心观察

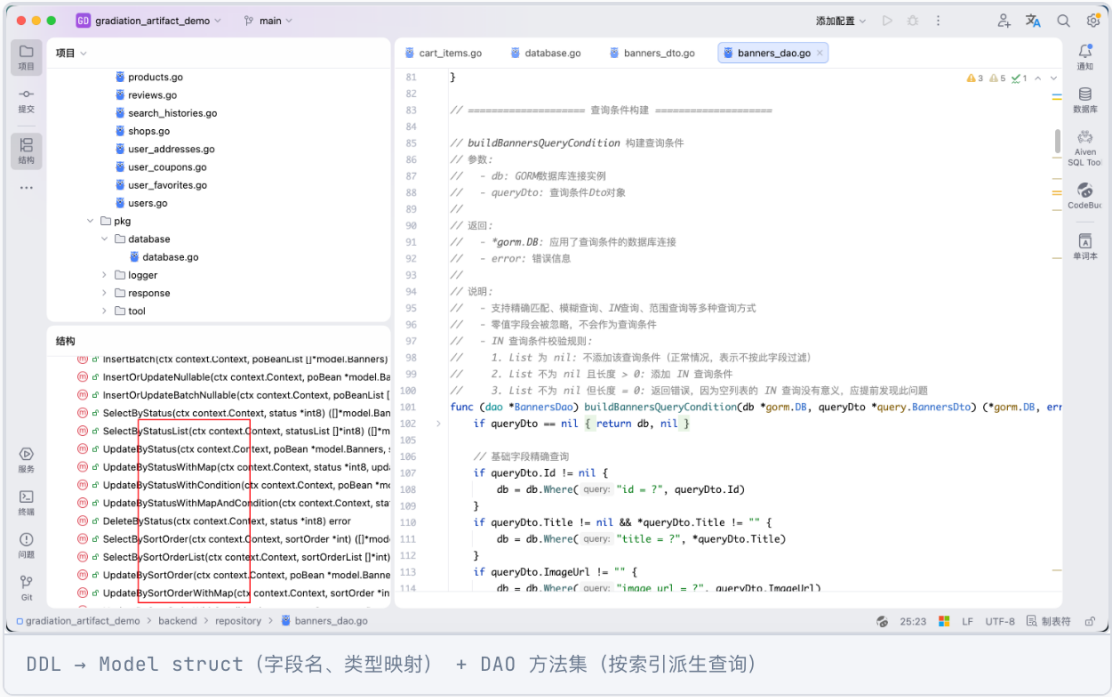
观察 · DDL 是数据层代码的充分输入

DAO 层和 Model 层的代码对业务的可操作独立发挥空间有限。只要拿到 DDL，就可以推断出：

- 索引情况 · 主键 → GetByPK，唯一索引 → GetByUK，普通索引 → GetByIdx
- 字段名 & 类型映射 · MySQL bigint → Go int64，varchar → string，可空 → 指针
- 查询场景 · 索引组合决定查询方法集，CRUD 是通用骨架
- 数据结构 · PO / DTO / VO 的字段完全由 schema 列决定

推论

既然 DDL 是充分输入，这些代码不需要 LLM 推理——交给确定性生成器（jen），封装为 Skill 按需加载。LLM 只负责需求理解 + 准备 schema + 验收产物。



右侧：一份 DDL 如何推导出 Model struct + DAO 方法集。

05 · 模板代码生成引擎 — jen 原理

① 输入

建表 SQL 或直连数据库。提取表名、列定义、索引定义作为**唯一事实源**。

② 解析

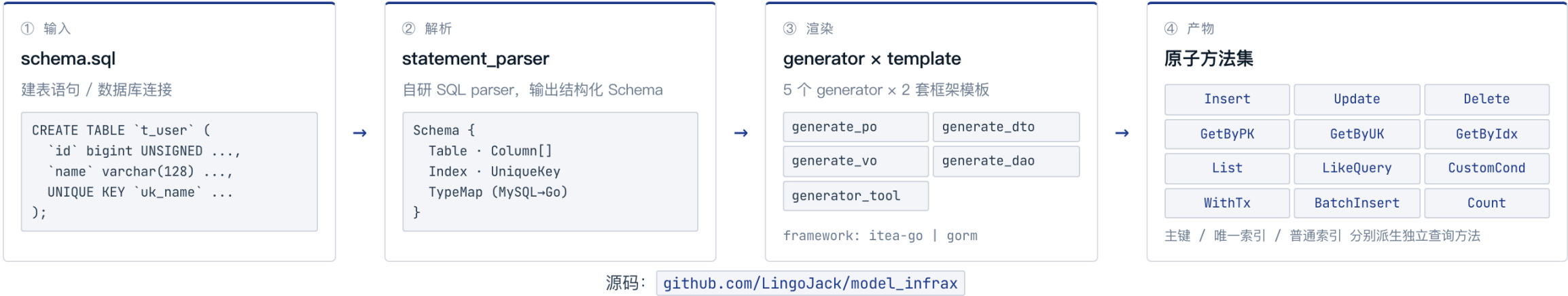
自研 SQL parser 提取**索引语义**——主键 / 唯一索引 / 普通索引——并做 MySQL→Go 类型映射和可空字段指针推导。

③ 渲染

结构化 Schema 拼入 Go template。5 个 generator 分别产出 PO/DTO/VO/DAO/Tool，支持 itea-go 和 gorm 两套框架。

④ 产物

按索引派生查询方法：主键→`GetByPK`，唯一索引→`GetByUK`，普通索引→`GetByIdx`。确定性输出，零漂移。



05 · 模板代码生成引擎 — jen 实测对比

验证方式

51 张表 DAO 全生成。同一份 schema.sql，对照：① Agent 逐文件写；② Agent 加载 jen Skill，调 jen + 验收。

速度更快，代码质量更高

两者都编译通过，差异在过程成本和字段精度。LLM 直写需反复"字段对齐→方法补全→编译修复"——长上下文维护本身成为负担。67 次 compact 说明上下文已频繁高水位。

字段漂移 · 系统性风险

11 个字段遗漏、4 个索引方法缺失——写到第 30 张表时 Agent 已忘了第 5 张表的唯一索引。jen 以 schema 为唯一事实源，从根本消除漂移。

结论

确定性生成器把规则稳定、易漂移的部分从 LLM 链路剥离，让 Agent 回归需求理解和集成验收。

维度	LLM 直接生成	jen + Skill	改善
编译通过	通过	通过	—
字段一致性	247 / 258 = 95.7%	258 / 258 = 100%	+4.3 pp
CRUD 覆盖率	100%	100%	—
索引查询覆盖	22 / 26 = 84.6%	26 / 26 = 100%	+15.4 pp
LLM 调用次数	582	11	↓ 53×
工具调用次数	2,006	15	↓ 134×
Token 消耗	24,219,892	483,775	↓ 50×
会话总耗时	53 min 5 sec	1 min 56 sec	↓ 27×
压缩触发次数	67	0	—
人工修正次数	0 (2 次网络重试不计)	0	—

论文表 7-8 / 7-9 · 51 张表 DAO 全生成 · 数据来源：metrics.jsonl + transcript.jsonl

05 · 模板代码生成引擎 — webapp-gen 全栈 workflow

设计思路

不是"AI 一次性写完一个 App", 而是一条11 步固化 workflow。把大模型不擅长的部分交给模板, 专注需求理解、契约定义、业务逻辑、增量集成。

三个关键设计

- 脚手架前置 · 模板保证结构完整性, 避免 AI 从零搭建框架
- 原型反馈闭环 · Step 4–5 先用 mock 出可视原型, 用户迭代确认后再写后端
- 三环境矩阵验收 · 本地 + 单容器 + 全栈 compose, 同一份测试脚本三环境都要过

jen Skill 嵌入

Step 6 写完 SQL schema 后调 @skill:jen 生成 DAO 层, 确定性部分从 LLM 链路剥离。



SECTION 06

总结与展望

四大贡献、开源仓库、未来方向。

github.com/LingoJack/jcli

jcli · 主项目

Rust · 5.2 万行 · 上下文治理 / 工具强化 / 多 Agent / Skill & Hook

github.com/LingoJack/model_infrax

jen · 模板代码引擎

Go · 自研 SQL parser + Go template · 封装为 Skill

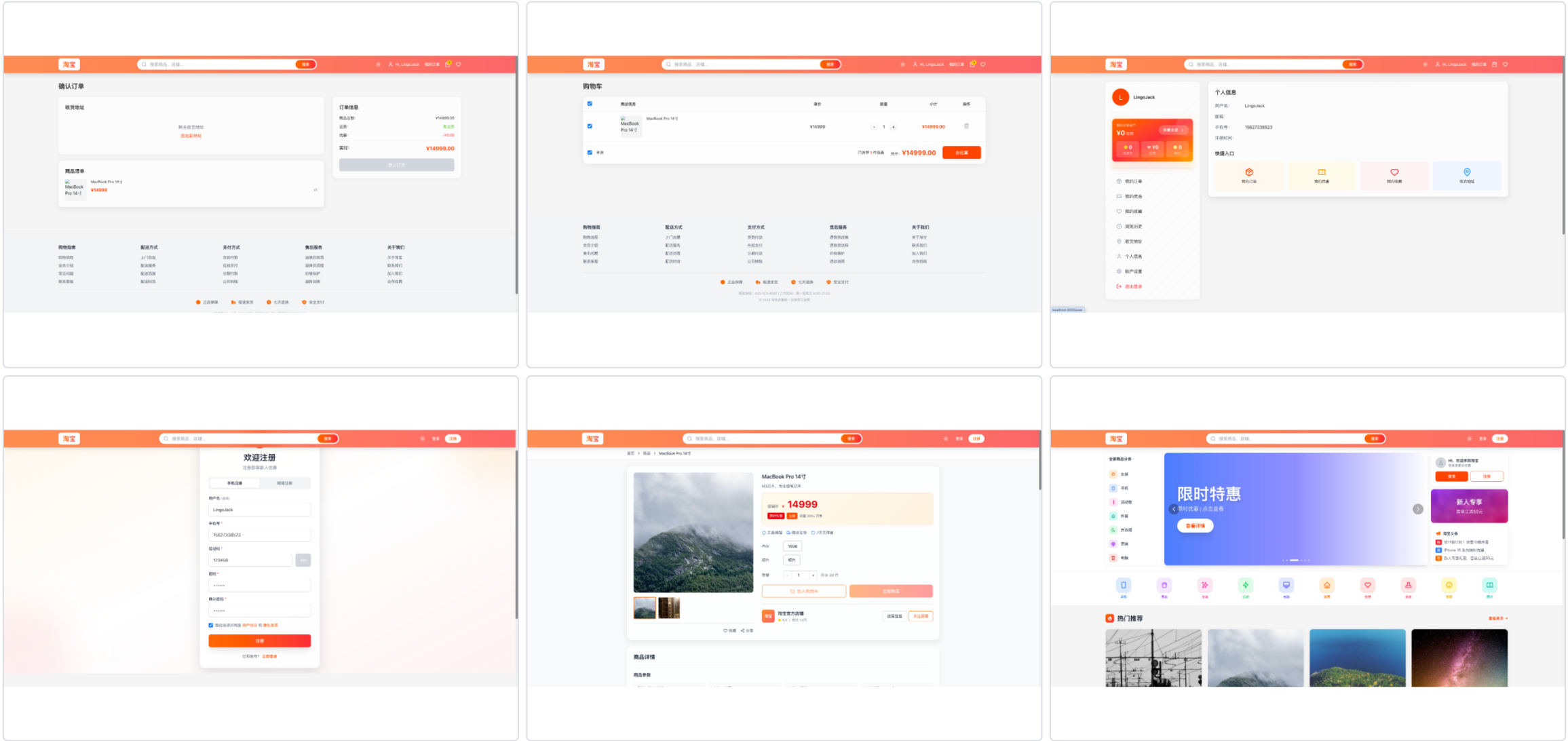
github.com/LingoJack/gradiation_artifact_demo

artifact_demo · 端到端验证

React + Go + GORM + Podman · webapp-gen 全流程生成 · 三环境验收通过

所有代码开源 · MIT License

06 · 总结与展望 — 交付制品预览



gradiation_artifact_demo · React + Go + GORM + Podman · 三环境验收通过

上下文治理与状态感知

分级消息窗口 + 多级压缩 + 豁免高价值消息 + 动态状态感知注入。让 Agent 在长任务中持续低幻觉运行。

面向场景强化的工具

Grep / Ask / Plan / Edit / 执行三态 / system-reminder / Hook。共同保障单 Agent 的长时间工程闭环。

多 Agent 协作机制

有了单 Agent 能力，多 Agent 协作需要消息同步与上下文隔离。Teammate + SubAgent + SendMessage Gate，三组实验全部通过。

模板代码生成引擎

即使有了多 Agent，业务软件中仍有规则确定但不适合 LLM 推理的部分。jen 以 schema 为事实源确定性生成，token ↓ 50x、耗时 ↓ 27x、字段一致性 100%。

THANKS

谢谢聆听

Q & A

github.com/LingoJack/jcli

github.com/LingoJack/model_infrax

github.com/LingoJack/gradiation_artifact_demo

Agent：通过上下文治理和工具调用，
沟通物理世界与逻辑世界。